

Lab Practice 6: P4 Telemetry — Multi-Hop Route Inspection and In-Switch Latency Measurement

This laboratory introduces two techniques for network telemetry implemented entirely in the P4 data plane. Both exercises go beyond forwarding: packets themselves carry measurement data that switches write at line rate, with no involvement from the control plane during measurement. The same compilation and deployment workflow from the previous laboratories applies: compile with `p4c-bm2-ss`, start the Mininet topology with `topo.py`, and install forwarding rules using `simple_switch_CLI`.

The guided exercise (MRI) implements a simplified variant of In-Band Network Telemetry (INT). Each switch traversed by a packet appends its own identifier and the current queue depth to a growing header stack carried inside the IP packet as an IPv4 option. The receiver can read the complete sequence of hops and their per-hop congestion state directly from the arriving packet, without querying any external monitoring system.

The student activity (MySec) implements a custom protocol for in-switch latency measurement. A packet follows a bounce path — h1 to s1 to s2 back to s1 and finally back to h1 — while switches record processing timestamps in dedicated header fields. The sender recovers the packet and reads the latency values without any out-of-band coordination. A key distinction from MRI is that MySec uses a custom IP protocol number (169) to gate the data-plane logic, whereas MRI relies on an IPv4 option type.

Exercise 1: MRI — Multi-Hop Route Inspection (Guided)

MRI is a simplified implementation of In-Band Network Telemetry (INT). INT is a framework defined by the P4.org consortium that allows network devices to embed monitoring data directly into data-plane packets at line rate, without generating separate control messages or copying packets to a collector. In its complete form, INT carries per-hop information such as switch ID, queue depth, ingress and egress timestamps, and link utilization. MRI implements the core idea: a growing header stack in which each switch inserts its ID and the current output queue depth.

The mechanism relies on IPv4 options. Each packet that carries MRI data has its `ihl` field set above 5 (the minimum for a plain IPv4 header). The option type 31 signals the presence of an MRI header, which itself contains a count field and a stack of switch records (`swid`, `qdepth`). Every switch that processes the packet in egress calls `push_front(1)` to grow the stack by one entry, writes its own identifier and queue depth, and increments `ihl`, `totalLen`, and `optionLength` accordingly. The receiver reads the full stack, which lists switches in reverse path order (last hop first), obtaining both the path that the packet traversed and the congestion state at each node.

The topology consists of three P4 switches and five hosts. Two hosts are connected to `s1` (subnet 10.0.1.0/24), two to `s2` (subnet 10.0.2.0/24), and one to `s3` (subnet 10.0.3.0/24). The link between `s1` and `s2` is rate-limited to 0.5 Mbps, acting as a congestion bottleneck that can be saturated with background traffic to produce observable queue depth values in the MRI records. The third switch provides an alternate route, allowing the receiver to distinguish different paths from the switch identifiers carried in the stack.

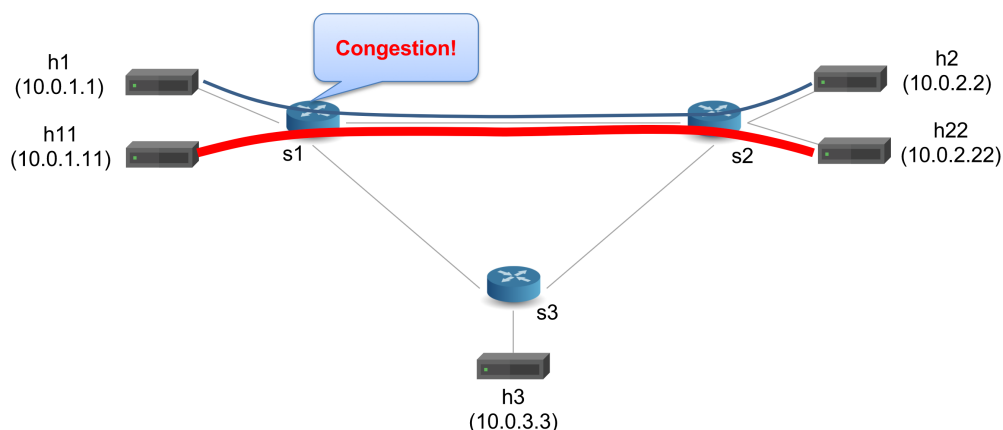


Figure 1: MRI topology: three P4 switches, five hosts, and a 0.5 Mbps bottleneck between `s1` and `s2`.

The host and port assignments are:

- `s1` (Thrift port 9090): `h1` at 10.0.1.1/24 on port 1, `h11` at 10.0.1.11/24 on port 2, `s2` on port 3 (bottleneck), `s3` on port 4.
- `s2` (Thrift port 9091): `h2` at 10.0.2.2/24 on port 1, `h22` at 10.0.2.22/24 on port 2, `s1` on port 3 (bottleneck), `s3` on port 4.

- s3 (Thrift port 9092): h3 at 10.0.3.3/24 on port 1, s1 on port 2, s2 on port 3.

The exercise uses L3 forwarding. Each host is configured with a default route to a virtual gateway IP, and the switch rewrites the Ethernet destination MAC at each hop. Static ARP entries are preconfigured in each host by topo.py so that cross-subnet traffic resolves correctly without ARP flooding. Standard tools such as ping produce standard IPv4 packets without MRI options; the provided send.py and receive.py scripts are required to create and parse the custom headers.

Step 1: Make the helper scripts executable and compile the P4 program. From the mri folder in the lab repository:

```
chmod +x send.py receive.py
```

```
mkdir -p p4src/build
```

```
p4c-bm2-ss --p4v 16 -o p4src/build/bmv2.json p4src/mri.p4
```

The compiler translates mri.p4 into bmv2.json. A deprecation warning about the output format may appear and can be ignored. Unlike the previous laboratories, this exercise does not require the `--p4runtime-files` flag, since the control plane uses `simple_switch_CLI` directly.

Step 2: Start the Mininet topology:

```
sudo python3 mininet/topo.py
```

The topology script starts three BMv2 instances (s1 on Thrift port 9090, s2 on port 9091, s3 on port 9092), creates five hosts with static ARP and gateway routes, and configures the 0.5 Mbps bottleneck link between s1 and s2. Leave this terminal open.

Step 3: In a second terminal, install the forwarding rules in all three switches. Each switch requires two types of entries: a default action for the egress swtrace table, and LPM entries for the ingress ipv4_lpm table.

```
simple_switch_CLI --thrift-port 9090 < s1-commands.txt
```

```
simple_switch_CLI --thrift-port 9091 < s2-commands.txt
```

```
simple_switch_CLI --thrift-port 9092 < s3-commands.txt
```

This exercise introduces a new CLI command: `table_set_default`. Unlike `table_add`, which installs a match entry, `table_set_default` sets the action that executes when no match entry exists — or, in this case, when the table has no key at all. The egress table `swtrace` has no match fields; every packet that reaches the egress pipeline with a valid MRI header triggers it unconditionally. The command assigns a unique switch identifier to each BMv2 instance:

```
table_set_default MyEgress.swtrace MyEgress.add_swtrace 1
```

Each command file also contains LPM entries for the `ipv4_lpm` table. `s1` and `s2` each receive four entries: two `/32` entries for their directly connected hosts, and two `/24` entries for the remote subnets forwarded through the inter-switch links. `s3` receives three entries: one `/32` entry for `h3` and two `/24` entries for the remote subnets reachable through `s1` and `s2`. The LPM entries follow the syntax introduced in the previous laboratories:

```
table_add MyIngress.ipv4_lpm MyIngress.ipv4_forward  
<ip>/<prefix> => <next-hop-mac> <port>
```

Step 4: Verify connectivity from the Mininet CLI:

```
mininet> pingall
```

The expected result is 20% dropped (16 of 20 pairs succeed). The four failing pairs are `h1–h11` (both on `s1` in subnet `10.0.1.0/24`) and `h2–h22` (both on `s2` in subnet `10.0.2.0/24`), in both directions. Because these host pairs share the same `/24` subnet, the Linux networking stack on each host attempts to resolve the destination IP via ARP directly, without routing through the gateway. The P4 switch does not respond to ARP requests, so resolution fails and ping cannot proceed. This is correct and expected behavior; all cross-switch pairs must succeed before proceeding.

Step 5: Open additional host terminals and run the MRI test. From the Mininet CLI:

```
mininet> xterm h1 h2 h11 h22
```

In the `h22` terminal, start a UDP server to absorb background traffic:

```
h22# iperf -s -u
```

In the h2 terminal, start the MRI receiver. This script uses Scapy with a custom IPOption_MRI dissector that parses the MRI option type and the SwitchTrace header stack:

```
h2# ./receive.py
```

In the h11 terminal, start a saturating background flow toward h22 to fill the bottleneck queue:

```
h11# iperf -c 10.0.2.22 -t 15 -u
```

Immediately after, in the h1 terminal, send MRI-tagged packets toward h2:

```
h1# ./send.py 10.0.2.2 "P4 is cool" 30
```

The send.py script constructs UDP packets that include an IPv4 option of type 31 with count=0 and an empty swtrace list. This signals to every switch on the path that the packet carries MRI data. Each switch appends its own record during egress processing. The receiver at h2 prints the decoded packet for each arriving frame using Scapy's pkt.show2() function. The relevant section of each printout is the options field, which shows the MRI structure:

```
got a packet
###[ MRI ]###
  option      = 31
  length      = 20
  count       = 2
  \swtraces   \
    |###[ SwitchTrace ]###
    |  swid      = 2
    |  qdepth    = 1
    |###[ SwitchTrace ]###
    |  swid      = 1
    |  qdepth    = 0
```

The count field shows how many switches inserted a record. The stack is ordered most-recent-first: the first entry (swid=2) was inserted by s2 (the last

switch before h2), and the second entry (swid=1) was inserted by s1. While the background flow from h11 saturates the s1–s2 bottleneck, the qdepth value for swid=1 should rise above zero, reflecting the growing output queue at s1. After the iperf flow ends and the queue drains, the qdepth values should return to zero or one.

Step 6: Verify path visibility on an alternate route. Open a host terminal for h3 and repeat the test on the path that crosses s3 instead of s2:

```
mininet> xterm h3
```

```
h3# ./receive.py
```

```
h1# ./send.py 10.0.3.3 "P4 is cool" 5
```

This time the path is h1 → s1 → s3 → h3, and the swtrace stack at h3 will show swid=3 (most recent) and swid=1. This contrast is the central observation of Multi-Hop Route Inspection: the header stack reveals which switches were actually traversed, without any external monitoring infrastructure. No single packet in this topology ever passes through all three switches simultaneously; the path difference is encoded directly in the switch identifiers carried inside the packet.

P4 Pipeline Overview

The P4 program (p4src/mri.p4) implements IPv4 LPM forwarding in the ingress pipeline and MRI stack manipulation in the egress pipeline. The program defines the MRI-specific headers alongside the standard Ethernet and IPv4 headers:

```
header ipv4_option_t {  
    bit<1>  copyFlag;  
    bit<2>  optClass;  
    bit<5>  option;  
    bit<8>  optionLength;  
}  
header mri_t {  
    bit<16> count;
```

```

}
header switch_t {
    switchID_t swid;
    qdepth_t qdepth;
}
struct headers {
    // ...
    switch_t[MAX_HOPS] swtraces;
}

```

The parser uses a two-state chain to consume the MRI data. The `parse_mri` state extracts the base MRI header, stores the count in parser metadata, and transitions to `parse_swtrace` if any records are present. The `parse_swtrace` state is recursive: it extracts one `switch_t` header per call, decrements the remaining counter, and transitions back to itself until the counter reaches zero:

```

state parse_mri {
    packet.extract(hdr.mri);
    meta.parser_metadata.remaining = hdr.mri.count;
    transition select(meta.parser_metadata.remaining) {
        0      : accept;
        default: parse_swtrace;
    }
}

state parse_swtrace {
    packet.extract(hdr.swtraces.next);
    meta.parser_metadata.remaining =
        meta.parser_metadata.remaining - 1;
    transition select(meta.parser_metadata.remaining) {
        0      : accept;
        default: parse_swtrace; // recursive
    }
}

```

The egress control defines the action `add_swtrace`, which prepends a new en-

try to the stack and adjusts the IPv4 header length fields to account for the eight additional bytes (4 for swid, 4 for qdepth):

```
action add_swtrace(switchID_t swid) {  
    hdr.mri.count = hdr.mri.count + 1;  
    hdr.swtraces.push_front(1); // grows the stack by one  
    hdr.swtraces[0].setValid();  
    hdr.swtraces[0].swid = swid;  
    hdr.swtraces[0].qdepth =  
        (qdepth_t) standard_metadata.deq_qdepth;  
    hdr.ipv4.ihl = hdr.ipv4.ihl + 2;  
    hdr.ipv4_option.optionLength =  
        hdr.ipv4_option.optionLength + 8;  
    hdr.ipv4.totalLen = hdr.ipv4.totalLen + 8;  
}
```

The table `swtrace` has no match key; it unconditionally executes `add_swtrace` via its default action when `hdr.mri` is valid. This is why the control plane installs the switch identifier using `table_set_default` rather than `table_add`. Because `ihl` and `totalLen` change on every packet, `MyComputeChecksum` recalculates the IPv4 checksum with the updated field values.

Activity 1: MySec — In-Switch Latency Measurement

MySec implements a custom in-switch latency measurement protocol. A packet constructed by the sender carries an 8-field header (`mysec_t`) and travels a pre-defined bounce path: $h1 \rightarrow s1 \rightarrow s2 \rightarrow s1 \rightarrow h1$. While in transit, `s1` records egress and ingress timestamps directly into the packet header on each of its two passes; `s2` acts as a pure reflector and does not modify the header. When the packet returns to `h1`, the sender reads the latency values from the arriving frame without any out-of-band communication.

The protocol is triggered by IP protocol number 169, a non-standard value with no real-world assignment. The data plane uses this value to identify MySec packets in the parser and ingress pipeline, routing them through the dedicated `TS_table` rather than the standard L2 forwarding table; only packets that carry a valid `mysec_t` header as a result of this classification can subsequently trigger

the timestamp logic in the egress pipeline. Standard tools that generate IPv4 traffic (ping, iperf, hping3) use different protocol numbers and are forwarded normally; only the provided send_mysec.py script generates the correct frame structure. The bounce path is enforced by the control plane: s2 is configured to redirect packets back toward s1 rather than delivering them to h2, regardless of the packet state at the time of arrival.

The topology consists of two P4 switches and two hosts arranged in a linear chain:

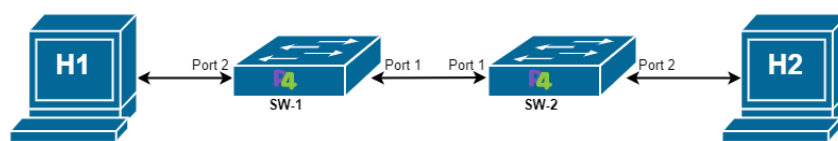


Figure 2: MySec topology: h1 and h2 connected through s1 and s2. A MySec packet enters s1 from h1, crosses to s2, is redirected back to s1, and is finally returned to h1.

- s1 (Thrift port 9090): h1 at 10.0.0.1/24 on port 1, s2 on port 2.
- s2 (Thrift port 9091): s1 on port 1, h2 at 10.0.0.2/24 on port 2.

All links are configured with 5 Mbps bandwidth, 5 ms delay, and 1% packet loss. Both hosts are in the same /24 subnet, and static ARP entries are populated by topo.py. Standard (non-MySec) traffic is forwarded by l2_exact_table using destination MAC as the sole match key. MySec traffic (proto = 169) bypasses l2_exact_table entirely and is handled by a separate two-key table (TS_table) that matches on both destination MAC and ingress port.

MySec Header

The mysec_t header occupies 304 bits (38 bytes) and contains eight fields:

```
header mysec_t {  
    bit<4>    ingress_port;  
    bit<4>    egress_port;  
    bit<48>   process_time_sw1;  
    bit<48>   process_time_sw2;  
    bit<48>   egress_time_sw1;
```

```
bit<48> ingress_back_time_sw1;
```

```
bit<48> total;
```

```
bit<48> th;
```

```
}
```

The fields `ingress_port` and `egres_port` are state-machine markers, not physical port numbers. They allow the egress pipeline of `s1` to distinguish whether the current processing pass is the forward leg (`h1` toward `s2`) or the return leg (`s2` toward `h1`). The six 48-bit fields store timing-related values, but their semantics differ: `egress_time_sw1` and `ingress_back_time_sw1` are absolute timestamps recorded from `standard_metadata.egress_global_timestamp` and `standard_metadata.ingress_global_timestamp` respectively; `process_time_sw1`, `process_time_sw2`, and `total` are processing-time durations derived from timestamp differences or sums, expressed in nanoseconds; and `th` is a 0/1 threshold flag that indicates whether the accumulated `total` exceeds a configured limit. All values are written in the egress pipeline, which is the first point in the `v1` model pipeline where both global timestamps are simultaneously available for a given packet transit. Note that despite the naming convention, `process_time_sw2` is computed by `s1` on the forward leg; the subscript reflects the direction of travel, not the physical switch performing the computation.

Packet Lifecycle

Figure 3 illustrates the conceptual timing model of the bounce path.

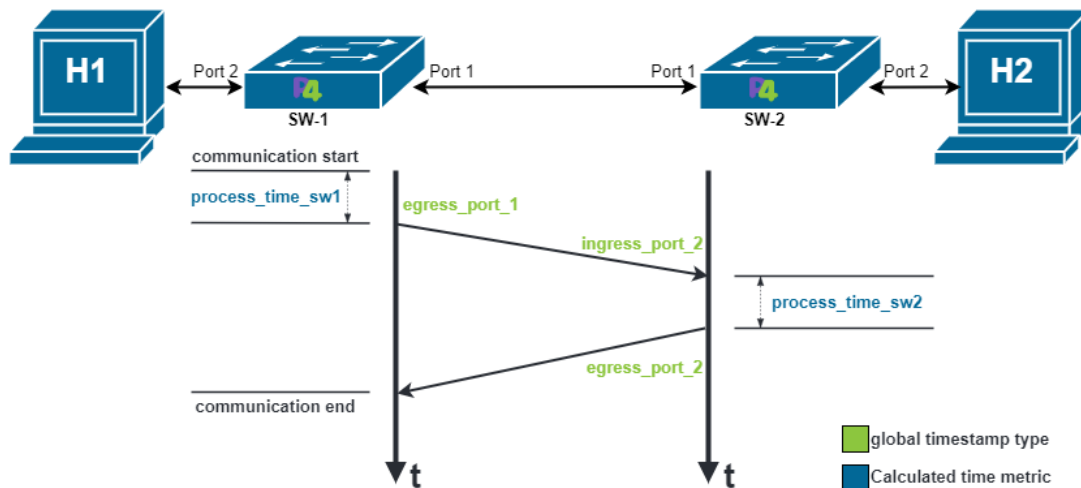


Figure 3: MySec timing diagram: bounce path $h1 \rightarrow SW-1 \rightarrow SW-2 \rightarrow SW-1 \rightarrow h1$. SW-1 writes timestamps on both passes. SW-2 acts as a pure reflector and does not compute any timestamp.

The initial packet sent by `send_mysec.py` carries `ingress_port = 1`, `egres_port = 2`, and zeroes in all timestamp fields. The packet enters `s1` on the forward leg (arriving on port 1) and is directed to port 2 (toward `s2`) by the ingress TS.table. In the egress pipeline, `s1` detects the forward-leg condition and computes the egress latency for that pass, writing it to `process_time_sw2`. It also updates the state-machine fields and rewrites the Ethernet destination MAC to prepare the packet for the return path.

The packet arrives at `s2`. The P4 implementation in `s2` does not write timestamps: it evaluates no timestamp conditions and writes nothing to the mysec header. It simply applies TS.table and redirects the packet back toward `s1` on port 1. The packet is never delivered to `h2`.

The packet re-enters `s1` on the return leg (arriving on port 2). In the egress pipeline, `s1` detects the return-leg condition and computes the egress latency for this pass, writing it to `process_time_sw1`. It also records the absolute egress timestamp and the three additional metrics from Part 2 of the TODO. The packet exits toward `h1` on port 1, and `send_mysec.py` captures the returning frame, printing all completed timestamp fields.

Forwarding Tables

The ingress apply block selects the forwarding table based on the IP protocol:

```
if (hdr.ipv4.protocol != PROTOCOL_MYSEC) {  
    l2_exact_table.apply();  
} else {  
    TS_table.apply(); // two-key: dst_MAC + ingress_port  
}
```

The TS_table requires two entries in s1 and two entries in s2. In s1, the first entry forwards MySec packets destined for h2_MAC arriving on port 1 to port 2 (toward s2, forward leg); the second entry forwards MySec packets destined for h1_MAC arriving on port 2 to port 1 (toward h1, return leg). In s2, both entries redirect to port 1 (back toward s1), because the bounce path never delivers a MySec packet to h2 directly. The two-key match (dst_MAC, ingress_port) is necessary because s1 processes the same packet twice: without the ingress_port key, the switch cannot determine whether a given MySec packet is on the outbound or return leg.

TODO Sections to Complete

Open p4src/main.p4 from the mysec folder in the lab repository. The program contains two TODO regions in the EgressPipeImpl control, both within the block that processes MySec packets.

The first TODO (**Exercise 3 TO-DO**) requires completing the port-based conditions that distinguish the two passes of s1. The egress control uses a local variable to represent the port number from which the packet arrived on the return leg. The comment instructs you to assign this variable the correct port number so that the conditional branch for the return leg activates on the second pass through s1. A second branch, structured as an else-if, handles the forward leg and must also reference the correct port. Refer to the timing diagram in the repository to identify which port each condition should check. The same TODO region also requires completing the rewrite of the Ethernet destination MAC address so that, after the forward-leg processing, the packet is directed back toward h1 and not toward h2.

The second TODO (**Exercise 3 TO-DO, Part 2 — Additional Metrics**) is nested inside the return-leg branch. Three lines are commented out, each containing a placeholder that must be replaced with an expression:

- `ingress_back_time_sw1`: record the absolute ingress timestamp of `s1` on the return path. Use the appropriate `standard_metadata` timestamp field available in the egress pipeline.
- `total`: compute the combined processing time across both directions. Use the two `process_time` fields already written in previous passes.
- `th`: evaluate whether the total exceeds a threshold of your choice and write 1 or 0 to this field. This implements a data-plane decision based on accumulated telemetry without controller involvement.

Do not modify the parser, the ingress control, the forwarding tables, the checksum block, or the deparser. The skeleton already provides the complete structure; only the expressions inside the TODO regions need to be filled in.

Compile and Deploy

Once the implementation is complete, compile the program from the `mysec` folder:

```
mkdir -p p4src/build
p4c-bm2-ss --p4v 16 -o p4src/build/bmv2.json \
--p4runtime-files p4src/build/p4info.txt p4src/main.p4
```

Start the Mininet topology:

```
sudo python3 mininet/topo.py
```

In a second terminal, install the forwarding rules in both switches:

```
simple_switch_CLI --thrift-port 9090 < s1-commands.txt
simple_switch_CLI --thrift-port 9091 < s2-commands.txt
```

Each command file contains two entries for `l2_exact_table` (standard L2 forwarding for ping traffic) and two entries for `TS_table` (context-aware forwarding

for MySec traffic). Verify that all entries are accepted without errors before proceeding.

Verify Connectivity and Run the MySec Test

Confirm that standard connectivity works before testing MySec:

```
mininet> pingall
```

The expected result is 0% dropped: h1 and h2 are in the same /24 subnet and share static ARP entries configured by topo.py, so standard ICMP traffic can reach both hosts without gateway routing. If either pair fails, review the `l2_exact.table` entries before proceeding.

Send one MySec packet from h1 and observe the result:

```
mininet> h1 python3 mininet/send_mysec.py
```

The script prints the outgoing packet (with all timestamp fields at zero) and the returning packet (with timestamps filled in by the switches). A successful run produces non-zero values in `process_time_sw1` and `process_time_sw2` and returns a formatted result block:

```
=====
MySec | In-switch latency results
=====
process_time_sw1      : XXXX ns
process_time_sw2      : XXXX ns
egress_time_sw1       : XXXXXXXXXXXX
ingress_back_time_sw1 : XXXXXXXXXXXX
total                 : XXXX
th                    : 0 or 1
=====
```

Typical values for `process_time_sw1` and `process_time_sw2` in a BMv2 software switch running on a virtual machine range from 500 to 50,000 nanoseconds. Both values must be significantly lower than 5,000,000 ns (the configured link delay) for the measurement to reflect switch processing time rather than link propagation delay.

//////

If the script reports that no response was received, verify that s2-commands.txt contains an entry for h1_MAC in TS_table. This entry is required because s1 modifies the Ethernet destination MAC before the packet reaches s2, and s2 must match the updated address to correctly redirect the packet back toward s1.

Discussion Questions

1. In MRI, the switch modifies ihl, totalLen, and optionLength on every packet that passes through egress. In MySec, no IPv4 header fields are modified by the forwarding or measurement pipeline. In both cases, however, the P4 programs include a checksum computation block. Explain which program requires an active checksum recalculation and why the other can leave that block empty or commented out.
2. MRI reads standard_metadata.deq_qdepth in the egress pipeline. MySec reads standard_metadata.egress_global_timestamp and standard_metadata.ingress_global_timestamp, also in egress. Explain why both measurements must be taken in egress rather than ingress, and identify what information would be unavailable or incorrect if the same logic were attempted in the ingress pipeline.
3. In the MRI exercise, a packet traveling from h1 to h2 produces a swtrace with count=2 and records for swid=1 and swid=2. A packet from h1 to h3 also produces count=2, but with records for swid=1 and swid=3. No single packet in this topology ever produces count=3. Explain what the term Multi-Hop Route Inspection implies about observability, and describe a topology change or routing policy that would result in count=3 for some flows.
4. The MySec protocol uses IP protocol number 169, which has no IANA assignment. MRI uses IPv4 option type 31. Both mechanisms are incompatible with standard routers and firewalls in production networks. Describe one production-grade mechanism standardized by the P4.org community that addresses the same telemetry goals while remaining compatible with existing IP infrastructure.

5. Both MRI and MySec are bounded by the size of a single packet: MRI is limited by MAX_HOPS (9 records), and MySec reports only two latency samples per round trip. If either mechanism were extended to a topology with tens of switches, what scalability constraints would arise with respect to packet-size growth, parser state complexity, and the correctness of the collected measurements?